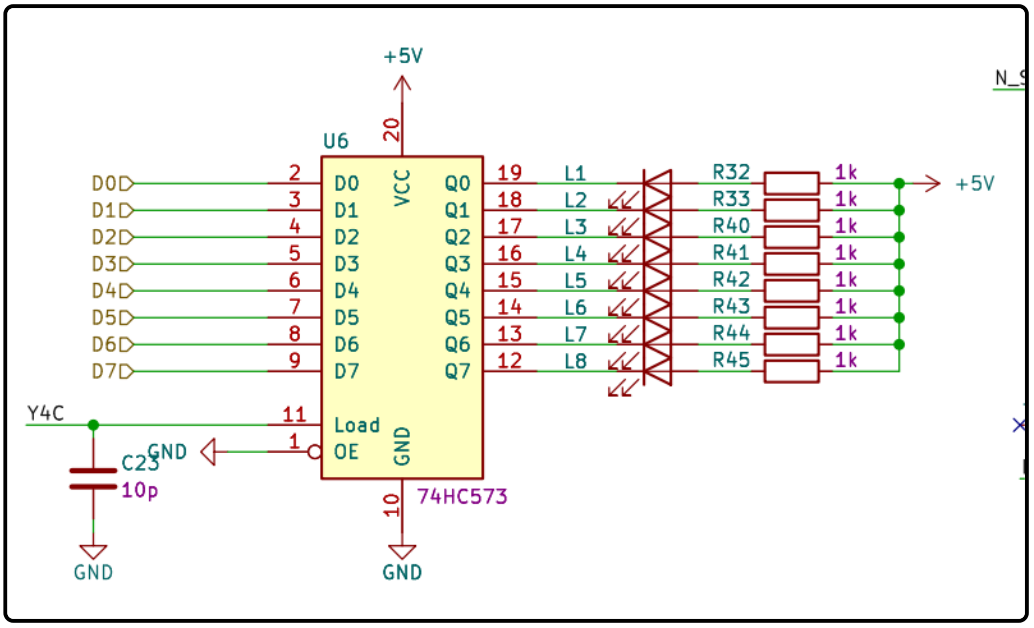
LED

LED底层原理

LED控制相对简单，需要注意的是LED给低电平(0)点亮，给高电平(1)熄灭，对应的锁存器地址为Y4C->0x80。



我们可以直接给P0口赋值，然后开启锁存器将数据输出。与数码管不同，LED可以一次性控制所有灯的状态。在底层实现中，我们接收整个ucLed数组，将每个元素按位组合成一个字节数据，然后对该数据取反后输出到P0口（因为为了操作方便，我们在逻辑上使用1表示点亮，但硬件上实际需要0才能点亮）。为了减少I/O操作，我们只在LED状态发生变化时才更新输出。另外还提供了一个关闭所有LED的函数，便于实现全局调光功能（PWM调光部分将在后文详述）。

```
// 保存当前LED状态，用于状态变化检测
idata unsigned char temp_1=0x00;
idata unsigned char temp_1_old=0xff;

/**
  * 功能：LED显示控制
  * 描述：根据传入的LED状态数组控制8个LED的亮灭
  *       只有当LED状态发生变化时才更新输出，减少I/O操作
  * 参数：ucLed - LED状态数组，每个元素对应一个LED
  *             0表示亮，1表示灭
  * 示例：{1,0,1,0,1,0,1,0} 对应二进制 10101010
  */
```

```c
void Led_Disp(unsigned char *ucLed)
{
    unsigned char temp;

    // 将8个LED状态合并为一个字节
    temp_1=0x00;
    temp_1=(ucLed[0]<<0)|(ucLed[1]<<1)|(ucLed[2]<<2)|(ucLed[3]<<3)|
        (ucLed[4]<<4)|(ucLed[5]<<5)|(ucLed[6]<<6)|(ucLed[7]<<7);

    // 只有当LED状态发生变化时才更新
    if(temp_1!=temp_1_old)
    {
        P0 = ~temp_1;  // LED低电平亮，所以需要取反
        //通过P2口地址线选择LED锁存器
        P2 = (P2&0x1f) | 0x80;  // 将P2的高3位设置为100，选择LED锁存器
        P2 &= 0x1f;             // 将P2的高3位清零，释放锁存器
        temp_1_old = temp_1;    // 保存当前状态
    }
}

/**
  * 功能：关闭所有LED
  * 描述：将所有LED熄灭
  */
void Led_Off()
{
    unsigned char temp;

    P0 = 0xff;  // LED低电平亮，所以全1表示全灭

    //通过P2口地址线选择LED锁存器
    P2 = (P2&0x1f) | 0x80;  // 将P2的高3位设置为100，选择LED锁存器
    P2 &= 0x1f;             // 将P2的高3位清零，释放锁存器

    temp_1_old=0x00;        // 重置状态记录
}
```

LED应用层

LED的应用层实现非常简单，只需为ucLed数组中的各元素赋值（1表示点亮，0表示熄灭），然后调用Led_Disp函数显示即可。（PWM调光功能将在后续PWM部分详细介绍）

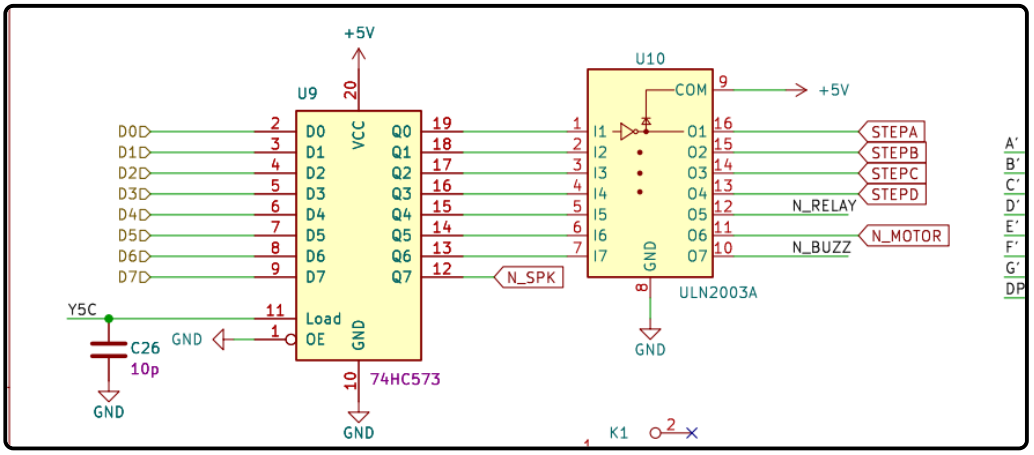


```c
pdata unsigned char ucLed[8] = {0, 0, 0, 0, 0, 0, 0, 0};
/**
 * 功能：LED显示处理函数
 * 描述：设置LED显示状态
 */
void Led_Proc()
{
    // 设置LED1-4亮(1表示亮)，LED5-8灭(0表示灭)
    ucLed[0] = 1;
    ucLed[1] = 1;
    ucLed[2] = 1;
    ucLed[3] = 1;
    ucLed[4] = 0;
    ucLed[5] = 0;
    ucLed[6] = 0;
    ucLed[7] = 0;
    Led_Disp(ucLed);
}
```

蜂鸣器、继电器和电机

蜂鸣器、继电器和电机底层实现

这三个外设共用同一个锁存器芯片，对应锁存器地址为Y5C->0xa0，因此将它们放在一起讨论。



由于这三个外设共用一个锁存器，在控制时必须确保各自的操作不会相互影响。实现方法是使用一个全局变量来维护所有外设的状态，启用某个外设时将对应位置1，关闭时将对应位置0。置1操作使用按位或运算(|=)，置0操作使用按位与非运算(&= ~)，这样可以精确控制单个位而不影响其他位的状态。

```c
// 保存当前外设状态，用于状态变化检测
idata unsigned char temp_2=0x00;
```

```c
idata unsigned char temp_2_old=0xff;
/**
  * 功能: 蜂鸣器控制
  * 描述: 控制蜂鸣器的开启和关闭
  * 参数: enable - 0表示关闭, 1表示开启
  */
void Beep(bit enable)
{
    unsigned char temp;

    // 设置蜂鸣器状态位(P0.6)
    if(enable) temp_2 |= 0x40;    // 将第6位置1, 开启蜂鸣器
    else temp_2 &= ~(0x40);       // 将第6位置0, 关闭蜂鸣器

    // 只有当状态发生变化时才更新
    if(temp_2!=temp_2_old)
    {
        P0 = temp_2;
        // 通过P2口地址线选择外设锁存器
        P2 = (P2&0x1f) | 0xa0;  // 将P2的高3位设置为101, 选择外设锁存器
        P2 &= 0x1f;             // 将P2的高3位清零, 释放锁存器
        temp_2_old = temp_2;    // 保存当前状态
    }
}

/**
  * 功能: 电机控制
  * 描述: 控制电机的开启和关闭
  * 参数: enable - 0表示关闭, 1表示开启
  */
void Motor(bit enable)
{
    unsigned char temp;

    // 设置电机状态位(P0.5)
    if(enable) temp_2 |= 0x20;    // 将第5位置1, 开启电机
    else temp_2 &= ~(0x20);       // 将第5位置0, 关闭电机

    // 只有当状态发生变化时才更新
    if(temp_2!=temp_2_old)
    {
        P0 = temp_2;
        // 通过P2口地址线选择外设锁存器
        P2 = (P2&0x1f) | 0xa0;  // 将P2的高3位设置为101, 选择外设锁存器
        P2 &= 0x1f;             // 将P2的高3位清零, 释放锁存器
        temp_2_old = temp_2;    // 保存当前状态
    }
}

```

```c
/**
  * 功能：继电器控制
  * 描述：控制继电器的开启和关闭
  * 参数：enable - 0表示关闭，1表示开启
  */
void Relay(bit enable)
{
    unsigned char temp;

    // 设置继电器状态位(P0.4)
    if(enable) temp_2 |= 0x10;    // 将第4位置1，开启继电器
    else temp_2 &= ~(0x10);       // 将第4位置0，关闭继电器

    // 只有当状态发生变化时才更新
    if(temp_2!=temp_2_old)
    {
        P0 = temp_2;
        // 通过P2口地址线选择外设锁存器
        P2 = (P2&0x1f) | 0xa0;  // 将P2的高3位设置为101，选择外设锁存器
        P2 &= 0x1f;             // 将P2的高3位清零，释放锁存器
        temp_2_old = temp_2;    // 保存当前状态
    }
}
```

蜂鸣器、继电器和电机应用层

这些外设的应用层实现非常直观，只需调用各自的控制函数并传入1（开启）或0（关闭）参数即可。下面是集成在LED处理函数中的示例：

```c
/**
 * 功能：LED显示处理函数
 * 描述：设置LED显示状态
 */
void Led_Proc()
{
    Beep(1);
    Relay(1);
    Motor(1);
}
```

PWM实现

原理说明

PWM（脉冲宽度调制）是一种通过调整数字输出信号的高低电平时间比例来控制功率传递的技术。其核心概念是占空比，即在一个完整周期内高电平持续时间与总周期时间的百分比。例如，对于一个频率为100Hz（周期为10ms）的PWM信号，若占空比为20%，则高电平持续2ms，低电平持续8ms；若占空比为50%，高低电平各持续5ms；若占空比为80%，则高电平持续8ms，低电平持续2ms。

这个信号有两种体现

1. motor或者p34直接输出
2. seg和led的调光，我们通常考到的是LED的调光

实际实现

下面我使用定时器0跑任务，定时器1跑pwm在motor的输出，定时器2跑串口

为什么要用定时器1跑pwm而不用定时器0呢，因为通常来说，我们pwm输出的频率都比较高，那么就意味着pwm的中断的时间肯定是比1ms更短的，所以我们如果用定时器0去跑中断的话，由于定时器0的优先级比定时器1更高，就会导致定时器1的中断一直进不去，那么就会导致我们任务直接死掉

LED调光

我这里以LED调光为例（数码管是一样的道理，但是数码管一般不会考这玩意，因为我们的数码管本身就是扫频出来的，1ms更新一次显示的位置），我们首先需要有一个compare值和一个period值，一个用来对比（也就是我们高电平的时间），另一个就是我们到用来进行累加的，我们注意一下就行，但是还是要说明一下，我们看清楚题目（LED的调光的档位，比如9省赛就有4个档位），我们很容易就想到使用3 6 9 12来进行控制他的占空比，从而就有不同的亮度，当然我们这里是随便写的，就10%~100%轮换，并且不需要太过细腻的梯度（所以我们就不用开启一个定时器1了，直接在定时器0里面1ms跑就好了），如果分的更细，比如35%的亮度，那么就需要去开一个定时器1（100us）了，因为定时器0（1ms）最多也就10%的精准，如果分成100份，那么就会导致我们人眼的余晖效应消失，看着就感觉在闪烁。

```
1 // PWM控制相关变量
```

```
idata unsigned char pwm_period = 10; // PWM周期(1ms)
idata unsigned char pwm_compare = 1; // PWM占空比(10%)
void Key_Proc()
{
    // 检查延时是否到达
    if (Key_Slow_Down < 10)
        return;
    Key_Slow_Down = 0;

    // 读取按键值并计算按下/松开事件
    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Val ^ Key_Old); // 检测按键按下事件
    Key_Up = ~Key_Val & (Key_Val ^ Key_Old);  // 检测按键松开事件
    Key_Old = Key_Val;                        // 保存当前按键值作为下次检测的旧值

    // 按键8处理(调整PWM占空比)
    if (Key_Down == 8)
        pwm_compare = (++pwm_compare) % 10; // 循环调整PWM占空比(0-9)
}
void Timer0_Init(void) // 1毫秒@12.000MHz
{
    AUXR &= 0x7F; // 定时器时钟12T模式
    TMOD &= 0xF0; // 设置定时器模式
    TL0 = 0x18;   // 设置定时初始值
    TH0 = 0xFC;   // 设置定时初始值
    TF0 = 0;      // 清除TF0标志
    TR0 = 1;      // 定时器0开始计时
    ET0 = 1;      // 使能定时器0中断
    EA = 1;       // 全局中断使能
}

void Timer0_Isr(void) interrupt 1
{
    // 各模块减速计数器递增
    Key_Slow_Down++;
    Seg_Slow_Down++;
    Seg_Pos = (++Seg_Pos) % 8; // 更新数码管位置

    // 数码管显示处理
    if (Seg_Buf[Seg_Pos] > 20)
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos] - ',', 1); // 带小数点
    else
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos], 0); // 无小数点

    // PWM输出处理
    if (++pwm_period >= 10)
```

```
48          pwm_period = 0; // 周期计数(0-9)
49      if (pwm_period < pwm_compare)
50          Led_Disp(ucLed); // PWM高电平期间点亮LED
51      else
52          Led_Off(); // PWM低电平期间熄灭LED
53  }
54
```

PWM控制Motor输出指定频率更改占空比的波

我们这里其实和LED一模一样，只是说我们这里通常使用的是一个1kHz的波形（13国赛），所以我们要单开一个定时器1（100us），我们将其分为10份，每一份的高低电平分别进行赋值，那么我们就可以得到不同占空比，

```
1   // PWM控制相关变量
2   idata unsigned char pwm_period = 10; // PWM周期(1ms)
3   idata unsigned char pwm_compare = 1; // PWM占空比(10%)
4   void Key_Proc()
5   {
6       // 检查延时是否到达
7       if (Key_Slow_Down < 10)
8           return;
9       Key_Slow_Down = 0;
10
11      // 读取按键值并计算按下/松开事件
12      Key_Val = Key_Read();
13      Key_Down = Key_Val & (Key_Val ^ Key_Old); // 检测按键按下事件
14      Key_Up = ~Key_Val & (Key_Val ^ Key_Old);  // 检测按键松开事件
15      Key_Old = Key_Val;                        // 保存当前按键值作为下次检测的旧值
16
17      // 按键8处理(调整PWM占空比)
18      if (Key_Down == 8)
19          pwm_compare = (++pwm_compare) % 10; // 循环调整PWM占空比(0-9)
20  }
21  /* 定时器1初始化，100微秒@12.000MHz */
22  void Timer1_Init(void) // 100微秒@12.000MHz
23  {
24      AUXR &= 0xBF; // 定时器时钟12T模式
25      TMOD &= 0x0F; // 设置定时器模式
26      TL1 = 0x9C;             //设置定时初始值
27      TH1 = 0xFF;             //设置定时初始值
28      TF1 = 0;        // 清除TF1标志
29      TR1 = 1;        // 定时器1开始计时
30      ET1 = 1;        // 使能定时器1中断
```

```
}

void Timer1_Isr(void) interrupt 3
{
    // PWM输出处理
    if (++pwm_period >= 10)
        pwm_period = 0; // 周期计数(0-9)
    if (pwm_period < pwm_compare)
        Motor(1);
    else
        Motor(0);
}
```

PWM控制Motor输出指定频率和占空比可变的波（困难）

为什么我在上面写了一个PWM输出指定频率占空比的波，下面还有一个可变呢，因为这两个考察的东西虽然是一样的，但是频率和占空比同时可变，对我们的要求就更加高了，我们可以这样进行代码书写

时序控制实现原理

系统通过定时器中断来精确控制高低电平的切换时机。关键在于我们使用了一个基本时间单位（10微秒）作为计数基础，然后通过累计计数来测量和控制电平持续时间。

1. PWM 参数计算方法

代码中 `Update_PWM_Timing()` 函数实现了 PWM 参数的计算逻辑：

```
void Update_PWM_Timing(void)
{
    unsigned long period_us = 1000000UL / Sent_Freq;     // 周期(微秒)
    High_Time = ((period_us * Duty_Cycle) / 100 + 5) / 10; // 高电平时间(10微秒单位)
    Low_Time = (period_us - High_Time * 10) / 10;        // 低电平时间(10微秒单位)
    // 立即更新状态，确保平滑过渡
    TR1 = 0;          // 暂停定时器1
    Motor(0);         // 先将输出置为低电平
    signal_level = 0; // 初始状态为低电平
    Timer_Count = 0;  // 计数器清零
    TR1 = 1;          // 重新启动定时器1
}
```

这个函数采用了以下计算步骤：

1. 计算周期时间（单位：微秒）= 1秒 ÷ 频率 = 1000000 ÷ Sent_Freq
2. 计算高电平持续时间 = (周期 × 占空比 ÷ 100 + 5) ÷ 10
   - 加 5 是为了四舍五入，提高计算精度
   - 除以 10 是为了转换为定时器的计数单位（10微秒）
3. 计算低电平持续时间 = (周期 - 高电平时间×10) ÷ 10
   - 确保高低电平之和等于完整周期，防止累积误差

2. 定时器中断控制 PWM 输出

定时器中断服务程序是实现 PWM 输出的核心部分：

```
void Timer1_Isr(void) interrupt 3
{
    // 每10us计时一次
    Timer_Count++;
    if (signal_level == 0)
    { // 当前为低电平阶段
        if (Timer_Count >= Low_Time)
        {
            Motor(1);        // 输出高电平
            signal_level = 1; // 切换到高电平阶段
            Timer_Count = 0;  // 重置计时器
        }
    }
    else
    { // 当前为高电平阶段
        if (Timer_Count >= High_Time)
        {
            Motor(0);        // 输出低电平
            signal_level = 0; // 切换到低电平阶段
            Timer_Count = 0;  // 重置计时器
        }
    }
}
```

此中断服务程序实现了一个状态机，在高低电平之间切换：

1. 每次中断（10微秒）计数器加1
2. 根据当前电平状态判断：
   - 低电平状态下，当计数达到 Low_Time 时，切换为高电平
   - 高电平状态下，当计数达到 High_Time 时，切换为低电平

3. 每次电平切换都重置计数器，开始下一阶段的计时